

PRODUCT REQUIREMENTS DOCUMENT

BlockCert

Privacy-Preserving Academic Record Registry

Version 5.0 | Final Year Undergraduate Project

React + TypeScript + Vite | Supabase Auth, PostgreSQL & Storage | Ethereum Sepolia | ethers.js v6 | Vercel

v5.0 NEW: Certificate file upload + Student photo stored in Supabase Storage. Both file hashes included in SHA-512 computation. Verifier sees full record, photo & certificate only after blockchain confirms authenticity.

This system does NOT generate certificate documents. It registers, stores, and verifies existing academic records and their supporting files.

1. Executive Summary

BlockCert is a decentralised application (dApp) designed to completely replace the slow, manual, email-based academic credential verification workflow used by universities today. Using a Hash-on-Chain architecture, it anchors a SHA-512 fingerprint of each student's complete academic record — including their certificate file and photo — on the Ethereum Sepolia blockchain, while keeping all personal data and files securely in Supabase.

Any employer or third party can verify a student's academic record, view their certificate, and confirm their identity photo in seconds — by scanning a QR code on a public portal. No login required. No email to the university. No waiting.

The Old Workflow BlockCert Replaces

Old Manual Workflow	BlockCert v5.0 Replacement
Company scans/photographs student's certificate	Student shares a QR code generated by BlockCert
Company emails scanned copy to the university	No email needed — verification is instant and self-serve
University staff manually checks CGPA in their records	Blockchain hash confirms authenticity in < 5 seconds
University writes a reply email to the	Verifier sees full record, photo, and certificate file

Old Manual Workflow	BlockCert v5.0 Replacement
company	directly on screen
Process takes days or weeks	Process takes under 30 seconds, 24/7, globally

Core Design Principles:

- Hash-on-Chain: The SHA-512 hash (computed from all record fields + certificate file + photo + secret salt) is the only thing stored on the blockchain. Zero personal data on-chain.
- File Storage via Supabase Storage: Certificate files and student photos are stored in private, access-controlled Supabase Storage buckets — not on IPFS or any public network.
- Blockchain as Gatekeeper: Verifiers can only access the full record, certificate, and photo after the blockchain confirms the hash is authentic.
- No Certificate Generation: BlockCert registers and verifies existing records — it does not produce certificate documents.
- Frontend-Only Architecture: React app communicates directly with Supabase and Sepolia. No custom backend server required.

2. Problem Statement

Problem	Impact
Paper and digital certificates can be forged or Photoshopped	Employers unknowingly hire unqualified candidates
Manual university verification via email takes days to weeks	Most employers skip verification; fraud goes undetected
University staff spend significant time on manual credential checks	Operational cost and delay for institutions
Existing blockchain solutions store PII on public ledgers	Privacy violations and regulatory non-compliance
No single, fast, globally accessible verification mechanism exists	Degree fraud continues to grow unchecked

The gap BlockCert fills:

A complete, end-to-end academic record registry that replaces manual university verification with an instant, tamper-proof, self-serve blockchain check — while keeping all sensitive student data and files private and secure.

3. Goals & Non-Goals

3.1 Goals

- Admin can register a complete student academic record: basic details, total marks, CGPA, certificate file (PDF/image), and student photo.
- Certificate file and student photo are stored securely in private Supabase Storage buckets with signed URL access.
- SHA-512 hash is computed from all record fields including certificate file hash and photo hash — any tampering with any part of the record invalidates verification.
- Only the final hash is stored on the Ethereum Sepolia blockchain via MetaMask.
- Full record including Supabase PostgreSQL data is saved off-chain.
- Student can log in, view their complete academic record, see their Etherscan link, and download a QR code.
- Verifier accesses the public portal (no login), scans/pastes a hash, and — only after blockchain confirms authenticity — sees the full record, student photo, and certificate file.
- Tampered records, forged QR codes, or altered files all produce a Verification Failed result.
- Supabase Auth manages all authentication with bcrypt. Default student credentials: roll number / roll number.
- Admin-reset passwords are shown exactly once and cannot be retrieved.

3.2 Non-Goals (Explicitly Out of Scope for v5.0)

- Generating or producing certificate documents — BlockCert only stores and verifies existing records.
- IPFS storage — student data is private; IPFS is a public network unsuitable for sensitive PII.
- Subject-wise or semester-wise marks breakdown (planned for future scope — see Section 17).
- Auto-fill form fields by scanning/OCR-reading an uploaded certificate (planned for future scope — see Section 17).
- A separate backend server (Python, Node, etc.).
- Mobile native apps (web-responsive is sufficient for v1).
- Payment or fee handling.

4. Users & Roles

Role	Auth	Key Responsibilities
University Admin	Supabase Auth (admin role) + MetaMask wallet on Sepolia	Enter student records, upload certificate file and photo, trigger hash computation, sign blockchain transaction, view all records, manage student credentials
Student	Supabase Auth (student	View own full academic record, view certificate

Role	Auth	Key Responsibilities
	role) Default: username = roll number, password = roll number	hash and Etherscan link, download QR code for sharing with employers
Verifier (Employer / Third Party)	No login — fully public portal	Scan QR or paste hash, receive blockchain confirmation, view full record including student photo and certificate file if authentic

4.1 User Stories

University Admin

"As a university admin, I want to enter a student's academic details, upload their degree certificate file and a photo, have the system compute a SHA-512 hash from all of this data, sign a Sepolia transaction via MetaMask, and save everything to Supabase — so the record is permanently anchored on-chain and accessible for instant verification."

Student

"As a student, I want to log in with my roll number, view my full academic record and certificate details, see proof of my blockchain transaction on Etherscan, and download a QR code I can hand to any employer as a self-contained verification tool."

Verifier (Employer)

"As an employer, I want to scan the student's QR code on a public portal, have the system confirm via blockchain that the record is authentic, and then immediately see the student's full academic record — including their photo and their degree certificate file — so I can complete verification in seconds without contacting the university."

5. System Architecture

5.1 Three-Layer Hybrid Model

Layer	Technology	What It Stores	Access Control
Blockchain (Trust Layer)	Ethereum Sepolia	SHA-512 hash only. No PII, no files, no names.	Public read. OnlyOwner write.
Database (Data Layer)	Supabase PostgreSQL	Full student record: name, roll no, dept, marks, CGPA, dates, hashes, storage paths	Admin: all records. Student: own record only (RLS). Verifier: by hash after blockchain confirms.
File Storage (Asset Layer)	Supabase Storage (private buckets)	Certificate file (PDF/image) and student photo. Accessed via short-lived	Private by default. Signed URL generated only after

Layer	Technology	What It Stores	Access Control
		signed URLs.	blockchain verification passes.

5.2 Why Supabase Storage (Not IPFS)

IPFS (InterPlanetary File System) is a public, decentralised network. Files uploaded to IPFS are accessible by anyone with the file's content hash (CID), with no access control and no deletion capability. Storing a student's degree certificate and photo on IPFS would expose sensitive personal data to the public — a serious privacy violation.

Supabase Storage provides:

- Private buckets — files are not publicly accessible by default.
- Row Level Security integration — same RLS policies that protect database records also protect files.
- Signed URLs — temporary, expiring access links generated server-side for verified requests only.
- Zero additional setup — already part of the Supabase stack used for Auth and PostgreSQL.

5.3 Key Source Files

File / Folder	Purpose
contracts/CertificateRegistry.sol	Solidity smart contract — on-chain hash registry
src/lib/crypto.ts	SHA-512 hashing: concatenates all fields + file hashes + university salt
src/lib/blockchain.ts	Blockchain abstraction: registerCertificate(), verifyCertificate()
src/lib/ethereum.ts	ethers.js v6 config: provider, MetaMask wallet connection, contract ABI
src/lib/storage.ts	Supabase Storage helpers: uploadCertificate(), uploadPhoto(), getSignedUrl()
src/pages/AdminDashboard.tsx	Admin hub: record list, add record, credential management
src/components/AddRecordForm.tsx	Core issuance: form, file upload, hash computation, MetaMask signing, Supabase insert
src/pages/VerifyPage.tsx	Public portal: hash input, blockchain query, conditional result display
src/integrations/supabase/	Supabase JS client and auto-generated TypeScript types

6. Data Model

6.1 On-Chain Data — Sepolia Smart Contract

The smart contract stores only a mapping of certificate hashes:

Field	Solidity Type	Description
hash (mapping key)	bytes32	SHA-512 hash of ALL record fields + certificate file hash + photo hash + university salt. The ONLY student-linked value on-chain.
exists	bool	True once registered. Used by verifyCertificate() to confirm authenticity.
timestamp	uint256	Block timestamp at time of registration.
blockNumber	uint256	Block number at time of registration.
issuer	address	University admin wallet that registered the hash.

What is NEVER stored on-chain: student name, roll number, marks, CGPA, dates, certificate file, student photo, or any PII.

6.2 SHA-512 Hash Computation ([src/lib/crypto.ts](#))

The hash is computed client-side by concatenating ALL of the following inputs, then running SHA-512. Any change to any input — including the files — produces a completely different hash and will fail verification:

Input	Type	Notes
student_name	Text field	Full name as entered by admin
roll_number	Text field	Unique student registration number
department	Text field	Academic department / programme
academic_year	Text field	Year of study
date_of_joining	Date field	Enrollment date
date_of_completion	Date field	Graduation / completion date
total_marks	Numeric field	Total marks obtained
cgpa	Numeric field	Cumulative Grade Point Average
certificate_file_hash	SHA-512 of file bytes	Hash of the uploaded certificate file — detects any tampering with the document
photo_hash	SHA-512 of file bytes	Hash of the uploaded student photo — detects photo substitution

Input	Type	Notes
UNIVERSITY_SECRET_SALT	Env variable	Server-side secret from .env — prevents brute-force hash guessing by external parties

Only the resulting 512-bit hex string is sent to the blockchain. All input fields, files, and the salt remain in Supabase.

6.3 Off-Chain Data — Supabase PostgreSQL

Table: certificates

Column	Type	Notes
id	uuid (PK)	Auto-generated primary key
student_name	text	Student full name
roll_number	text (unique)	Unique identifier; used as Auth username
department	text	Academic department
academic_year	text	Year of study
date_of_joining	date	Enrollment date
date_of_completion	date	Graduation date
total_marks	numeric	Total marks obtained
cgpa	numeric	Cumulative Grade Point Average (e.g. 8.7)
certificate_hash	text	Final SHA-512 hash — mirrors the on-chain value
blockchain_tx_hash	text	Sepolia tx hash — generates the Etherscan link
certificate_file_path	text	Supabase Storage path for the degree certificate file
photo_path	text	Supabase Storage path for the student photo
supabase_user_id	uuid (FK)	References auth.users — links record to student login
created_at	timestampz	Record creation timestamp

Row Level Security (RLS) Policies:

- Admins: full read/write on all records and storage objects.
- Students: read-only on their own record (WHERE supabase_user_id = auth.uid()).
- Verifiers: no direct Supabase database access. After blockchain confirmation, a backend Supabase function returns the record and generates signed URLs for the files.

6.4 File Storage — Supabase Storage Buckets

Bucket	Contents	Access Policy
certificates	Degree certificate files (PDF or image, max 10MB)	Private. Signed URL (15-minute expiry) generated only after blockchain hash verification passes.
photos	Student passport-style photos (JPG/PNG, max 2MB)	Private. Signed URL generated alongside certificate URL after verification passes.

File naming convention:

```
certificates/{roll_number}/{uuid}.pdf
photos/{roll_number}/{uuid}.jpg
```

7. Smart Contract Design

Contract: CertificateRegistry.sol | **Network:** Ethereum Sepolia Testnet

Address configured in: src/lib/ethereum.ts as VITE_CONTRACT_ADDRESS

7.1 Storage Structure

```
struct Certificate { bool exists; uint256 timestamp; uint256 blockNumber;
address issuer; }
mapping(bytes32 => Certificate) private certificates;
```

7.2 Functions

Function	Visibility	Description
registerCertificate(bytes32 hash)	onlyOwner	Stores a new hash. Reverts if hash already exists. Emits CertificateRegistered event.
verifyCertificate(bytes32 hash)	public view	Returns (exists, timestamp, blockNumber, issuer). Gas-free read call — no MetaMask needed.

7.3 Access Control

- onlyOwner: only the university admin wallet can call registerCertificate.
- Anyone can call verifyCertificate — no authentication, no gas cost.
- No delete or update functions — registered hashes are permanent and immutable.
- Duplicate hashes are rejected — each unique record is registered exactly once.

8. Technical Stack

Layer	Technology	Detail
UI Framework	React 18 + TypeScript	Component-based, fully type-safe
Build Tool	Vite	Fast HMR dev server and optimised production builds
Styling	Tailwind CSS	Utility-first responsive design
UI Components	shadcn/ui (Radix UI)	Accessible, composable component primitives
Animations	Framer Motion	Cyber-Grid themed animations and page transitions
Forms & Validation	React Hook Form + Zod	Schema-based validation including file type and size rules
Routing	React Router DOM	Client-side routing: /, /admin, /student, /verify
Blockchain Network	Ethereum Sepolia Testnet	Public testnet — real network conditions, no real ETH required
Smart Contract	Solidity ^0.8.x	CertificateRegistry.sol in /contracts
Web3 Library	ethers.js v6	MetaMask connection, contract calls, event listening
Wallet	MetaMask (browser extension)	Admin signs registerCertificate transactions. Required for issuance only.
Hashing	Browser Web Crypto API (SHA-512)	All hashing done client-side in src/lib/crypto.ts before any network call
Auth	Supabase Auth	Bcrypt password hashing, session management, role-based access
Database	Supabase PostgreSQL	Off-chain student record storage with RLS
File Storage	Supabase Storage	Private buckets for certificate files and student photos; signed URL access
Supabase Client	@supabase/supabase-js	Unified client for Auth, Database, and Storage
Deployment	Vercel	Live at block-cert-eight.vercel.app
Package Manager	Bun / npm	bun.lock and package-lock.json both present in repo

9. User Flows

9.1 Admin Flow — Full Record Issuance

- Admin logs into BlockCert via Supabase Auth (admin role).
- Admin connects MetaMask wallet to Sepolia testnet.

3. Admin fills the Add Record form with student details: Name, Roll Number, Department, Academic Year, Date of Joining, Date of Completion, Total Marks, CGPA.
4. Admin uploads the student's degree certificate file (PDF or image, max 10MB).
5. Admin uploads the student's photo (JPG/PNG, max 2MB).
6. React Hook Form + Zod validates all fields and file types/sizes before proceeding.
7. Certificate file and photo are uploaded to private Supabase Storage buckets. Storage paths are returned.
8. `src/lib/crypto.ts` computes SHA-512 hashes of both uploaded files individually.
9. `src/lib/crypto.ts` concatenates all text fields + both file hashes + `UNIVERSITY_SECRET_SALT` and computes the final master SHA-512 hash.
10. `src/lib/blockchain.ts` calls `registerCertificate(masterHash)` on Sepolia via `ethers.js`. MetaMask prompts admin to sign and pay gas (test ETH).
11. Transaction is confirmed on Sepolia. Transaction hash returned.
12. Supabase client inserts the full record into the certificates table (all fields + `certificate_hash` + `blockchain_tx_hash` + file storage paths).
13. Supabase Auth creates a student user account with default credentials (roll number / roll number).
14. Admin success screen shows: master hash, Etherscan link, and student default credentials with one-time copy warning.

9.2 Student Flow — View & Share Record

15. Student logs in using roll number as username and default password.
16. Student Dashboard displays: full academic record (name, dept, year, marks, CGPA), certificate hash, Etherscan transaction link, and blockchain status.
17. Student downloads their QR code (encodes the verification URL + master hash).
18. Student shares QR with employer as a self-contained verification tool.

9.3 Verifier Flow — Instant Public Verification (Replaces University Email)

19. Verifier opens `/verify` — no login required.
20. Verifier scans student's QR code or manually pastes the certificate hash.
21. `ethers.js` calls `verifyCertificate(hash)` on Sepolia — gas-free read call, no MetaMask needed.
22. Smart contract returns: `exists (bool)`, `timestamp`, `blockNumber`, `issuer address`.
23. If `exists = false`: red Verification Failed banner shown. Process ends.
24. If `exists = true`: Supabase query fetches the full student record using the hash as the lookup key.
25. Supabase Storage generates signed URLs (15-minute expiry) for the certificate file and student photo.
26. Green Authentic result displayed showing: student name, roll number, department, academic year, marks, CGPA, blockchain metadata (block number, timestamp, issuer), student photo, and a View Certificate button opening the signed URL.

9.4 Tamper Detection — Comprehensive Coverage

The Hash-on-Chain design covers all possible tampering vectors:

Tampering Attempt	Detection Mechanism	Result
Change any text field (e.g. raise CGPA from 7.1 to 9.5)	Recomputed hash differs from on-chain hash	Verification Failed
Swap the certificate file for a different/edited document	Certificate file hash changes, master hash changes	Verification Failed
Replace the student photo with a different person's photo	Photo hash changes, master hash changes	Verification Failed
Forge a QR code with a made-up hash	Hash does not exist in smart contract mapping	Verification Failed
Delete/modify the Supabase record after registration	On-chain hash still exists; recomputed hash from altered data will not match	Verification Failed

10. Screen Specifications

Screen / Route	Auth Required	Key Elements
Home /	None	System intro explaining the three user roles. Three CTAs: Admin Login / Student Login / Verify Now. Framer Motion Cyber-Grid background animation.
Admin Dashboard /admin	Supabase Auth (admin role) + MetaMask	Add Record button, full records table (all students), credential reset panel, Verify Blockchain Integrity button, recent activity feed.
Add Record Form (modal/page)	Admin	Text fields: Name, Roll No, Department (dropdown), Academic Year, Joining Date, Completion Date, Total Marks, CGPA. File uploads: certificate file (PDF/image, max 10MB), student photo (JPG/PNG, max 2MB). On success: master hash, Etherscan link, one-time credentials panel with copy warning.
Student Dashboard /student	Supabase Auth (student role)	Academic record details, certificate hash, Etherscan transaction link, blockchain status badge, QR code preview, Download QR button.
Verification Portal /verify	None — fully public	Hash input field or QR scan. Valid result: green Authentic panel with full record, student photo, View Certificate button (signed URL), block number, timestamp, issuer address. Invalid: red Verification Failed with explanation.

11. Authentication & Credential Policy

11.1 Supabase Auth

Supabase Auth handles all authentication. Passwords are bcrypt-hashed automatically. The application never accesses plaintext passwords after they are set.

11.2 Student Default Credentials

- Username: roll number (e.g. RA2211003010001)
- Password: roll number (same value — intentional default for first-time ease)
- Students are encouraged to change their password after first login.

11.3 Admin Password Reset for Students

- Admin clicks Reset Password on a student record in the Admin Dashboard.
- System generates a secure random password via the Supabase Auth Admin API.
- New plaintext password is displayed once in a highlighted box with a mandatory copy warning.
- Navigating away from this screen permanently destroys the plaintext — Supabase stores only the bcrypt hash.
- No recovery is possible — a second reset is the only option if the password is lost.

UI Warning on reset screen:

This password is shown only once and cannot be retrieved. Copy it now and share it with the student before leaving this page.

11.4 Admin Credentials

Admin accounts are provisioned directly in Supabase Auth. Admins also require a MetaMask wallet connected to Sepolia. These are two independent credentials — Supabase login authenticates the admin in the app; MetaMask signs blockchain transactions.

12. Integration & Routing

12.1 Client-Side Routes

Route	Auth Guard	Purpose
/	None	Landing page

Route	Auth Guard	Purpose
/admin	Supabase session (admin role)	Admin Dashboard
/student	Supabase session (student role)	Student Dashboard
/verify	None	Public Verification Portal
/verify?hash=<hash>	None	Pre-filled verification from QR scan

12.2 Supabase Operations

Operation	Service	Triggered By
Upload certificate file	Supabase Storage (certificates bucket)	Admin on form submit, before hashing
Upload student photo	Supabase Storage (photos bucket)	Admin on form submit, before hashing
INSERT full student record	Supabase PostgreSQL (certificates table)	Admin after blockchain tx confirmed
CREATE student user	Supabase Auth	Admin after Supabase record inserted
SELECT own record	Supabase PostgreSQL (RLS filtered)	Student on dashboard load
SELECT record by hash	Supabase PostgreSQL	Verifier portal after on-chain confirmation
Generate signed URL (certificate)	Supabase Storage	Verifier portal after record fetched
Generate signed URL (photo)	Supabase Storage	Verifier portal after record fetched
RESET student password	Supabase Auth Admin API	Admin on credential reset

12.3 Blockchain Calls (ethers.js v6)

Call	Contract Function	Direction	MetaMask Required
Register master hash	registerCertificate(bytes32)	Write — state-changing	Yes — admin signs
Verify hash	verifyCertificate(bytes32)	Read — view function	No — gas-free

13. Security & Privacy Considerations

Concern	Mitigation
PII on public blockchain	No personal data is ever sent to the blockchain. Only the SHA-512 master hash is on-chain.
Certificate file or photo exposed publicly	Stored in private Supabase Storage buckets. Only accessible via short-lived signed URLs (15-min expiry) generated after blockchain verification passes.
Hash brute-forcing	UNIVERSITY_SECRET_SALT in .env ensures external parties cannot reverse-engineer record details by guessing common names and roll numbers.
Certificate file tampering	Certificate file hash is an input to the master SHA-512 hash. Any change to the file changes the master hash, failing on-chain verification.
Photo substitution	Student photo hash is an input to the master hash. Swapping the photo changes the master hash, failing verification.
Unauthorised hash registration	onlyOwner modifier on registerCertificate — only the university's wallet can write to the contract.
Supabase data access control	RLS ensures students see only their own record. Admins have elevated privileges. Verifiers access data only through a controlled post-verification query.
SQL injection	Supabase JS client uses parameterised queries by default.
Student credentials	Default password = roll number. Admin-reset passwords shown once only; bcrypt hash stored by Supabase Auth.
Environment variables	VITE_SUPABASE_URL, VITE_SUPABASE_ANON_KEY, VITE_CONTRACT_ADDRESS, VITE_UNIVERSITY_SALT — stored in .env, never committed to Git.
Sepolia testnet in production	Sepolia is appropriate for demo. Production would require Ethereum mainnet or a permissioned chain with a formal security audit.

14. Dependencies (package.json)

Package	Purpose
react + react-dom	UI framework
typescript	Type-safe JavaScript
vite	Dev server and production build tool
tailwindcss	Utility-first CSS
shadcn/ui + @radix-ui/*	Accessible UI component library
framer-motion	Animations — Cyber-Grid theme and page transitions

Package	Purpose
react-hook-form	Form state management including file upload fields
zod	Schema validation — text fields and file type/size rules
react-router-dom	Client-side routing
ethers (v6)	Ethereum interaction — MetaMask, contract read/write
@supabase/supabase-js	Unified Supabase client: Auth, PostgreSQL, Storage
lucide-react	Icon library
react-qr-code or qrcode	Client-side QR code generation

15. Development Setup

15.1 Environment Variables (.env)

```
VITE_SUPABASE_URL=https://<your-project>.supabase.co
VITE_SUPABASE_ANON_KEY=<your-anon-key>
VITE_CONTRACT_ADDRESS=<deployed CertificateRegistry address on Sepolia>
VITE_UNIVERSITY_SALT=<secret salt for SHA-512 hashing>
```

15.2 Supabase Storage Setup

- Create two private buckets in Supabase Storage: certificates and photos.
- Set bucket policies to private (no public access).
- Configure RLS policies so only authenticated admins can upload; signed URL generation is restricted to server-side calls post-verification.

15.3 Steps to Run

27. Clone: `git clone https://github.com/amallal2004/BlockCert`
28. Install: `npm install` (or `bun install`)
29. Create `.env` with all four variables above.
30. Set up Supabase: create the certificates table (schema in Section 6.3) and both Storage buckets.
31. Deploy `CertificateRegistry.sol` to Sepolia via Remix or Hardhat. Copy the contract address to `.env`.
32. Run: `npm run dev`
33. Connect MetaMask to Sepolia and fund admin wallet with test ETH (`faucet.sepolia.dev`).

16. Demo Script (For Presentation)

- Open the home page. Explain what BlockCert does and compare it to the old university email verification workflow.
- Login as Admin. Show the dashboard and MetaMask connected to Sepolia.
- Add a student record. Fill all text fields, upload a real certificate PDF and a photo. Submit and watch the MetaMask signing popup.
- Show the success screen. Point out the master SHA-512 hash, the Etherscan link, and the one-time credentials panel with the copy warning.
- Open Sepolia Etherscan. Paste the tx hash. Show that only the hash is on-chain — no names, no marks, no files.
- Open Supabase Dashboard. Show the certificates table (full record) and the Storage buckets (certificate file and photo). Contrast with the minimal on-chain data.
- Login as Student. Show the dashboard with the full record, Etherscan link, and QR code. Download the QR.
- Open /verify. Scan or upload the QR. Show the green Authentic result with the student photo, View Certificate button, block number, and timestamp.
- Click View Certificate. Show the degree certificate file opening via the signed URL.
- Tamper test. Modify one character in the hash field, paste into /verify. Show the red Verification Failed result.
- Explain why it works. Walk through the hash computation including file hashes, the smart contract mapping lookup, and blockchain immutability.
- Demo password reset. Show the one-time display and navigate away to prove it is gone permanently.

17. Future Enhancements

17.1 Detailed Marks (Near-term)

- Subject-wise marks: store each subject name, marks obtained, maximum marks, and grade as a JSON array in the certificates table.
- Semester-wise breakdown: group subject marks by semester for a complete academic transcript view.
- Both would be displayed on the verifier result page alongside the existing CGPA.

17.2 Auto-Fill from Certificate Scan (AI/OCR — Future Scope)

When the admin uploads a certificate file, the system will automatically extract and pre-fill the Add Record form fields using OCR and AI parsing. The admin then reviews and confirms before submission.

Planned implementation:

- Admin uploads the certificate file in the first step.

- System sends the file to an OCR service (e.g. Google Vision API, AWS Textract, or a self-hosted Tesseract instance via a Supabase Edge Function).
- Extracted text is parsed to identify and map fields: name, roll number, department, dates, marks, CGPA.
- Form fields are pre-filled with extracted values. Admin corrects any errors and confirms.
- This eliminates manual data entry for routine record registration and reduces human error.

Note: Auto-fill is pre-fill only — the admin always reviews and submits. The system never registers data without explicit admin approval.

17.3 Other Planned Enhancements

- Production deployment on Ethereum mainnet or a university-consortium permissioned chain.
- Replace MetaMask dependency with a Supabase Edge Function holding a server-side private key — removing the need for admin to have a browser wallet.
- Multi-university support — each institution gets its own issuer address and storage namespace.
- Batch CSV upload — admin uploads a spreadsheet of students; system processes and registers all records in one session.
- Student self-service password reset via Supabase email OTP.
- Email/SMS to student when their record is added, delivering default credentials securely.
- Analytics dashboard — daily verifications, department breakdowns, active issuers.

18. Success Metrics

Metric	Target
Record issuance time (form submit to blockchain confirmed)	< 30 seconds (Sepolia ~12s block time)
Verification time (hash lookup + Supabase fetch + signed URL generation)	< 5 seconds
File upload success rate (certificate + photo)	100%
False positive rate (forged record marked authentic)	0%
False negative rate (valid record marked failed)	0%
QR code generation success rate	100%
Personal data written to blockchain	0 fields — master hash only
Certificate files exposed without verification	0 — signed URLs generated only post blockchain confirmation
Concurrent users supported (demo)	5+ users

Metric	Target
University emails sent for verification	0 — fully replaced by BlockCert

This PRD reflects BlockCert v5.0 as implemented in github.com/amallal2004/BlockCert. Production deployment requires a security audit, mainnet migration, formal key management policy, and data protection compliance review.